

TDD

domingo, 5 de octubre de 2025

10:15

Recordar que esta clase no tiene clase grabada, ya que es tema nuevo

Material de la bibliografía:

¿Qué es TDD (Desarrollo Dirigido por Tests)?

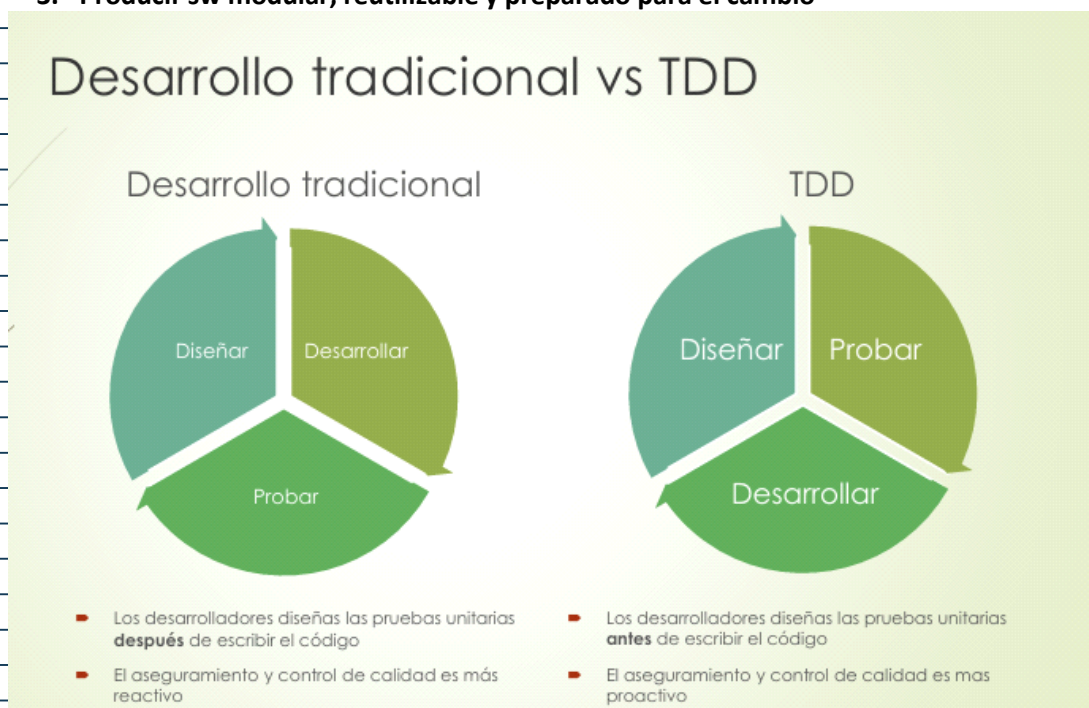
Es una técnica de diseño e implementación de software que se practica mediante la dirección del desarrollo a través de pruebas automatizadas. Es un componente clave de la metodología programación extrema.

Primariamente es una herramienta de diseño, un mecanismo de análisis y una técnica que abarca todas las actividades de desarrollo.

Objetivo principal de TDD es tener un código limpio que funcione.

Pilares de TDD

1. Implementar solo lo que el cliente necesita
2. Minimizar el nro de defectos en el sw en la fase de producción
3. Producir sw modular, reutilizable y preparado para el cambio



La imagen significa que el desarrollo tradicional y el desarrollo dirigido por pruebas (TDD)

La diferencia principal entre ambos enfoques radica **en cuando se escriben las pruebas unitarias** y como esto afecta el ciclo de diseño, la implementación y la calidad de sw

En el desarrollo tradicional (si se usa el ciclo de vida de cascada) las fases se siguen una tras otra. Probar recién se hace después de desarrollar, una vez que ya se ha generado el código. Es decir se realizan las pruebas después del diseño y la codificación. Aca el control de prueba es más reactivo, es decir que se detecta más al final de de las etapas, cuando el programa ya esta operativo.

En cambio, en TDD, es parte de la filosofía ágil como programación extrema (XP) a. TDD se enfoca en guiar el desarrollo con pruebas automatizadas.

El proceso de TDD es iterativo y se enfoca en un ciclo continuo que generalmente se describe como Rojo/Verde/Refactorizar:

. **Rojo (Diseñar/Probar):** Se escribe un pequeño test que *falle* (quizás ni siquiera compile al principio), lo que sirve como especificación del requisito. Esto se alinea con la primera fase de **Diseñar/Probar** en su esquema [Query].

2. **Verde (Desarrollar/Probar):** Se hace que la prueba funcione rápidamente, escribiendo el código *mínimo* necesario para superarla. Esto engloba las fases de **Probar/Desarrollar** en su esquema, asegurando que se cumpla la funcionalidad exigida por la prueba fallida.
3. **Refactorizar (implícito en el ciclo):** Se elimina toda la duplicidad creada en el proceso de hacer funcionar la prueba. Aunque no está explícitamente listado en los pasos secuenciales de su consulta, la refactorización es un pilar esencial del algoritmo TDD.

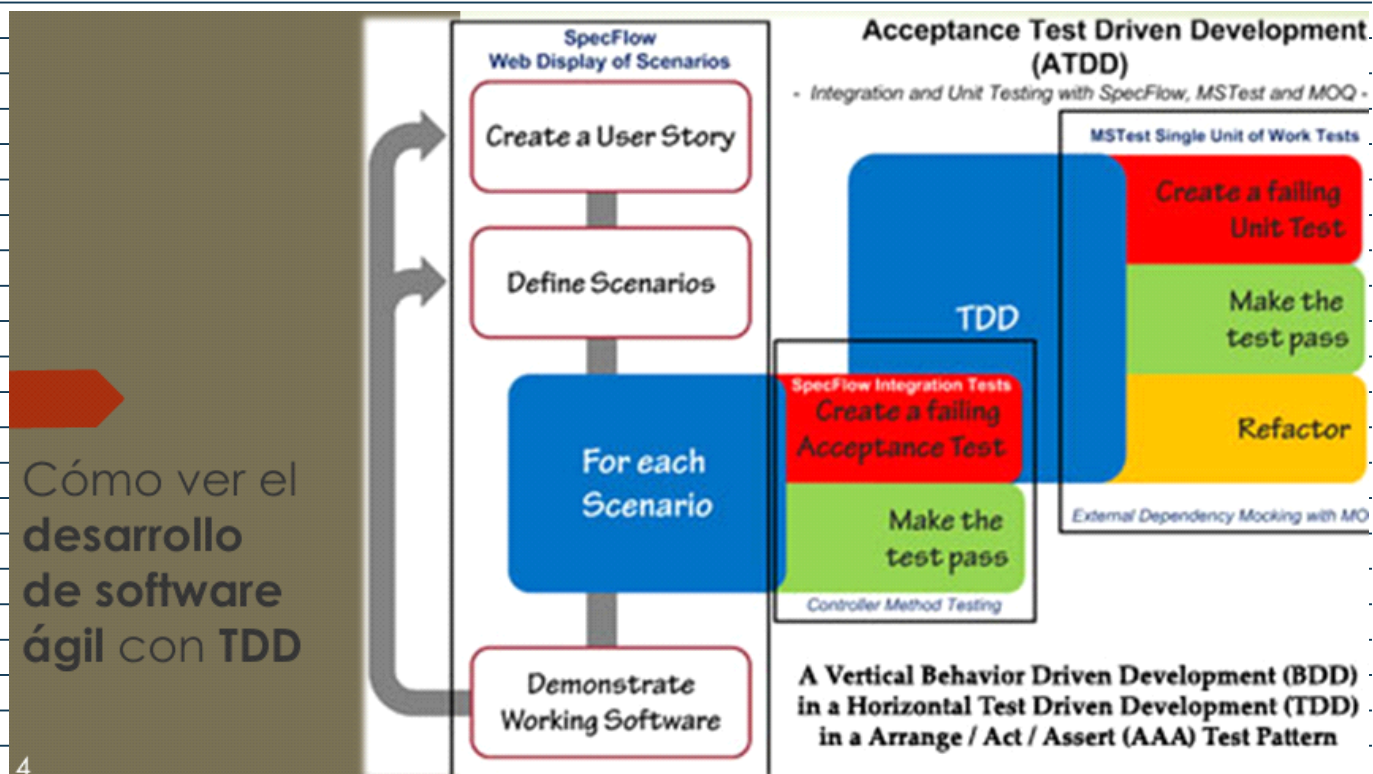
Desde <https://notebooklm.google.com/notebook/7bf30da7-180b-4cf3-9146-e3ae0ad3aecf>

Algunas de las características claves es que los desarrolladores **diseñan la pruebas unitarias antes de escribir el código**

Los tests y el código de producción se crean de forma conjunta, con las pruebas unos segundos antes que el código.

Acá el control de calidad es mas proactivo

primero escribís la prueba (test) que define *qué querés que el código haga*, **antes** de escribir el código.



Esto describe cómo **comienza el proceso** de desarrollo ágil con TDD y BDD:

1. Crear una user story
2. Definir escenarios concretos (casos de uso, codiciones específicas)

Quiero que el test falle al principio para ver si es útil, es decir para ver si detecta el error. Si no detecta errores, entonces no es un test útil.

Es una técnica avanzada que involucra otras dos prácticas: Escribir las pruebas primero (Test First Development) y Refactorización (Refactoring).

Para escribir las pruebas generalmente se utilizan las pruebas unitarias

Primero, que es una prueba unitaria?

Una **prueba unitaria** es una prueba que verifica **una parte muy pequeña del código**, normalmente **una sola función, método o clase**, de manera **aislada** del resto del sistema.

El objetivo de las pruebas unitarias es asegurarse de que cada parte individual del programa funciona correctamente por si misma.

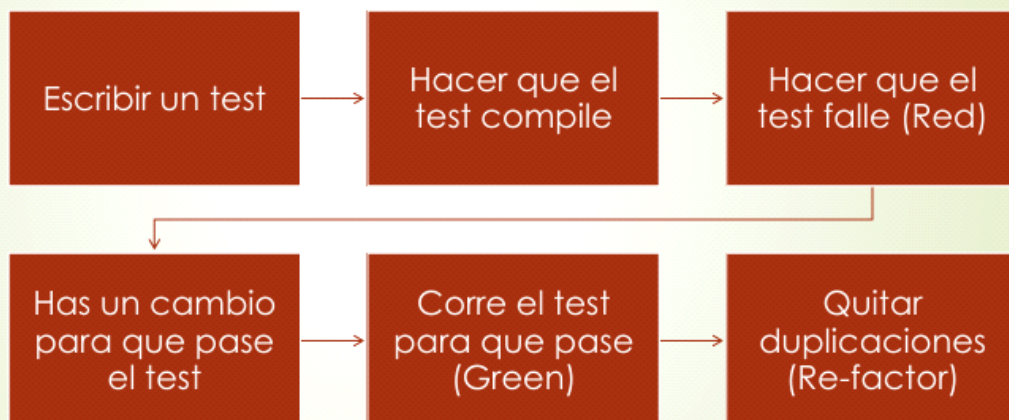
El TDD funciona gracias a las pruebas unitarias ya que se escribe el test unitario que falla (pq aun no se implementa el código).

Beneficios ♦ Código robusto ♦ Código más predecible y limpio, que permite saber cuándo el código está terminado ♦ Código más tolerable al cambio, al tener escritas las pruebas es más fácil de entender ♦ Código más seguro ♦ Aumenta la posibilidad de reducir la duplicación de código (si el test pasa el código ya existe) ♦ Código más barato de mantener, al ser más entendible ♦ Con la práctica acelera la velocidad de desarrollo y la posibilidad de despliegue continuo ♦ Asegura cobertura de prueba, lo que conlleva a código de calidad ♦ Obliga a pensar en cómo usar el componente, por lo tanto, el diseño de las API ♦ Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento

TDD tiene 3 leyes:

1. No escribir código hasta que el test case falle.
2. Solo se necesita un test que falle (tipo así es suficiente)
3. No se escribe más código en producción que el necesario para q

Pasos del Desarrollo conducido por Testing (TDD)

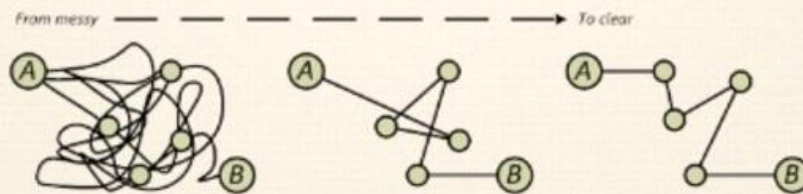


Refactoring

- Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos.
- Es una transformación del software que:
 - Preserva la estructura externa
 - Mejora la estructura interna

Son cambios que se realizan en el sistema que:

- No cambian el comportamiento observable (todas las pruebas aún pasan)
- **Eliminan la duplicación o la complejidad innecesaria**
- **Mejoran la calidad del software**
- Hacen que el código sea más simple y más fácil de entender
- Flexibiliza el código
- Hace que el código sea más fácil de cambiar



¿Por qué es necesario hacer Refactoring?

- Evitar el "deterioro del diseño"
- Limpiar desorden en el código
- Simplificar el código
- Incrementar la legibilidad y la comprensibilidad
- Encontrar errores
- Reducir el tiempo de depuración
- Incorporar el aprendizaje que hacemos sobre la aplicación
- Rehacer las cosas es fundamental en todo proceso creativo

¿Cómo hacer Refactoring?



Fuentes y Bibliografía ♦ Test Driven Development by example. Ken Beck ♦ Clean Code. Robert C. Martin ♦ Artículo Embracing Test Driven Development. Brad Huett ♦ Material introductorio para consultar: <https://www.youtube.com/watch?v=1gttkO9JKtU>